

LINQ Contains Method in C# with Examples

In this article, I will discuss the **LINQ Contains Method in C#** with Examples. Please read our previous article discussing the [LINQ Any Method in C#](#) with Examples. As part of this article, we will discuss the following pointers.

1. [What is LINQ Contains Method in C#?](#)
2. [Example to Understand LINQ Contains Method in C# using Primitive Type Collection.](#)
3. [Example to Understand LINQ Contains Method in C# using String Type Collection.](#)
4. [LINQ Contains Method with Complex Type in C#](#)
5. [Using IEqualityComparer Comparer with LINQ Contains Method in C#](#)
6. [Overriding Equals\(\) and GetHashCode\(\) Methods](#)
7. [Implementing IEquatable<T> Interface](#)
8. [When should you use the LINQ Contains Method in C#?](#)

What is LINQ Contains Method in C#?

The LINQ Contains Method in C# checks if a sequence (like a list or array) contains a specific element using the default equality comparer or a specified compare. It returns a boolean value indicating whether the element exists in the sequence. The method returns a Boolean value: true if the element is found in the collection and false otherwise.

The LINQ Contains Method in C# can be used with any collection (like an array, list, etc.) implementing the `IEnumerable<T>` interface. Three Contains Methods are available in C# and implemented in two different namespaces. They are as follows.

System.Collections.Generic namespace

```
public bool Contains(T item);
```

System.Linq namespace

```
public static bool Contains<TSource>(this IEnumerable<TSource> source, TSource value);  
public static bool Contains<TSource>(this IEnumerable<TSource> source, TSource value,  
IEqualityComparer<TSource> comparer);
```

- The Contains Method belongs to the **System.Collections.Generic** namespace takes one element as an input parameter; if that element is present in the data source, it returns true. Otherwise, it returns false.
- Two overloaded versions are available for the Contains method that belongs to the **System.Linq** namespace and one of the overloaded versions take **IEqualityComparer** as a parameter.

Example to Understand LINQ Contains Method in C# using Primitive Type Collection.

Let us see an example of using LINQ Contains Method in C# using Method and Query Syntax. In the following example, we check whether element 33 is present in the collection. The following example returns true as the data source (i.e., `IntArray`) contains element 33.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] IntArray = { 11, 22, 33, 44, 55 };
            //Using Method Syntax
            var IsExistsMS = IntArray.Contains(33);
            //Using Query Syntax
            var IsExistsQS = (from num in IntArray
                             select num).Contains(33);
            Console.WriteLine($"Is Element 33 Exist: {IsExistsMS}");
            Console.ReadKey();
        }
    }
}

```

Output: **Is Element 33 Exist: True**

Example to Understand LINQ Contains Method in C# using String Type Collection.

Let us see an example of using the LINQ Contains Method in C# using String type collection. For a better understanding, please look at the following example, which shows how to use the Contains Method in C# with String type collection. In the example below, we check whether the string collection contains the name Anurag. The following example will return False as the sequence or data source contains no Anurag element.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<string> namesList = new List<string>() { "James", "Sachin", "Pam", "Sara" };
            //Using Method Syntax
            //This method belongs to System.Collections.Generic namespace
            var IsExistsMS1 = namesList.Contains("Anurag");
            //This method belongs to System.Linq namespace
            var IsExistsMS2 = namesList.AsEnumerable().Contains("Anurag");
            //Using Query Syntax
            var IsExistsQS = (from num in namesList
                             select num).Contains("Anurag");
            Console.WriteLine($"Is Name Anurag Exist: {IsExistsQS}");
            Console.ReadKey();
        }
    }
}

```

```

    }
}
}

```

Output: **Is Name Anurag Exist: False**

Points to Remember:

Internally, the Contains method iterates over the collection, using the specified or default equality comparer to compare each element with the value. As soon as a match is found, it returns true. If the end of the collection is reached without finding the element, it returns false.

The performance of the Contains method depends on the collection's structure and size. For example, if the collection is a list or an array, the method performs a linear search, resulting in $O(n)$ complexity, where n is the number of elements in the collection. The performance could be better for collections with more efficient search capabilities (like a `HashSet<T>`).

Example to Understand LINQ Contains Method with Complex Type in C#:

Let us see an example of using the LINQ Contains Method with Complex Data Type in C# using both Method and Query Syntax. We are going to work with the following Student. So, create a class file named **Student.cs** and copy and paste the following code. As you can see, the Student class has three properties: ID, Name, and TotalMarks.

```

namespace LINQDemo
{
    public class Student
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public int TotalMarks { get; set; }
    }
}

```

The following example returns True as the object that we pass to the Contains method exists in the data source. The point you need to remember is while working with complex types, the Contains method checks the object reference, not the object's values. In the below example, the object reference of the object that we passed to the Contains method is available in the data source, so in this case, it returns true.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<Student> students = new List<Student>();

```

```

        };

        var student2 = new Student() { ID = 102, Name = "Preety", TotalMar
students.Add(student1);
students.Add(student1);
//Using Method Syntax
var IsExistsMS = students.Contains(student1);

//Using Query Syntax
var IsExistsQS = (from num in students
                  select num).Contains(student1);
Console.WriteLine(IsExistsMS);
Console.ReadKey();
    }
}
}

```

Output: True

Now, let us create a new student object with the existing student object property value and pass that newly created student object to the LINQ Contains method. For a better understanding, please have a look at the following example. The following example will return False even though the values we passed are available in the data source. This is because the default compare, used by the LINQ Contains Method, does not check the values; rather, it checks the object reference, and in this case, the object references are different.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<Student> students = new List<Student>()
            {
                new Student(){ID = 101, Name = "Priyanka", TotalMa
            },
                new Student(){ID = 102, Name = "Preety", TotalMar
            };
            //Using Method Syntax
            var IsExistsMS = students.Contains(new Student() { ID = 101, Name
"Priyanka", TotalMarks = 275 });
            var student1 = new Student() { ID = 101, Name = "Priyanka", TotalM
        };

            //Using Query Syntax
            var IsExistsQS = (from num in students
                              select num).Contains(student1);
            Console.WriteLine(IsExistsMS);
            Console.ReadKey();
        }
    }
}

```

```

    }
}

```

Output: **False**

As we already discussed in our previous articles, there are four ways in which we can tell the comparer to check the value equality instead of the reference equality. Let us see all those four ways with examples.

Using IEqualityComparer Comparer with LINQ Contains Method in C#:

In this approach, we need to create a class, and then we need to implement the **IEqualityComparer** interface. We need to provide the implementation for the Equals and GetHashCode methods. So, create a class file named **StudentComparer.cs** and copy and paste the following code.

```

using System.Collections.Generic;
namespace LINQDemo
{
    public class StudentComparer : IEqualityComparer<Student>
    {
        public bool Equals(Student x, Student y)
        {
            //If both object references are equal then return true
            if(object.ReferenceEquals(x, y))
            {
                return true;
            }
            //If one of the object reference is null then return false
            if (x is null || y is null)
            {
                return false;
            }
            return x.ID == y.ID && x.Name == y.Name && x.TotalMarks == y.TotalMarks;
        }
        public int GetHashCode(Student obj)
        {
            //If obj is null then return 0
            if(obj is null)
            {
                return 0;
            }
            int IDHashCode = obj.ID.GetHashCode();
            int NameHashCode = obj.Name == null ? 0 : obj.Name.GetHashCode();
            int TotalMarksHashCode = obj.TotalMarks.GetHashCode();
            return IDHashCode ^ NameHashCode ^ TotalMarksHashCode;
        }
    }
}

```

Then, you need to use the overloaded version of the Contains method, which takes **IEqualityComparer** as a parameter. Now, create an instance of the **StudentComparer** and pass that instance to the Contains

method, as shown in the below example.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<Student> students = new List<Student>()
            {
                new Student(){ID = 101, Name = "Priyanka", TotalMa
            },
                new Student(){ID = 102, Name = "Preety", TotalMarl
            };
            //Createing Student Comparer Instance
            StudentComparer studentComparer = new StudentComparer();
            //Using Method Syntax
            var IsExistsMS = students.Contains(new Student() { ID = 101, Name
            "Priyanka", TotalMarks = 275 }, studentComparer);
            var student1 = new Student() { ID = 101, Name = "Priyanka", TotalM
            };
            //Using Query Syntax
            var IsExistsQS = (from num in students
                               select num).Contains(student1, studentComparer);
            Console.WriteLine(IsExistsMS);
            Console.ReadKey();
        }
    }
}
```

Output: **True**

Overriding Equals() and GetHashCode() Methods within the Student Class

This is the second approach, and in this approach, we need to override the Equals and GetHashCode() in the Student class. By default, any type (predefined or user-defined) in the .NET Framework is inherited from the Object class. That means the Student class is also inherited from the Object class. So, we can override the Equals() and GetHashCode() methods of the Object class within the Student class. So, modify the Student class as shown below.

```
namespace LINQDemo
{
    public class Student
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public int TotalMarks { get; set; }
    }
}
```

```

        public override bool Equals(object obj)
        {
            //As the obj parameter type is object, so we need to
            //cast it to Student Type
            return this.ID == ((Student)obj).ID && this.Name == ((Student)obj).Name &&
this.TotalMarks == ((Student)obj).TotalMarks;
        }
        public override int GetHashCode()
        {
            return this.ID.GetHashCode() ^ this.Name.GetHashCode() ^
this.TotalMarks.GetHashCode();
        }
    }
}

```

Next, we need to modify the Main method of the Program class, as shown below. Here, we need to use the overloaded version of the Contains method, which does not take any parameter.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<Student> students = new List<Student>()
            {
                new Student(){ID = 101, Name = "Priyanka", TotalMarks = 275},
                new Student(){ID = 102, Name = "Preety", TotalMarks = 275}
            };
            //Using Method Syntax
            var IsExistsMS = students.Contains(new Student() { ID = 101, Name = "Priyanka", TotalMarks = 275 });
            var student1 = new Student() { ID = 101, Name = "Priyanka", TotalMarks = 275 };
            //Using Query Syntax
            var IsExistsQS = (from num in students
                             select num).Contains(student1);
            Console.WriteLine(IsExistsMS);
            Console.ReadKey();
        }
    }
}

```

Output: **True**

Implementing IEquatable<T> Interface in Student Class

This is the third approach, and in this approach, we need to implement the `IEquatable<T>` Interface in Student Class. We need to implement the `Equals` Method of the `IEquatable<T>` Interface, and we also need to override the `GetHashCode` method of the `Object` class. So, modify the `Student` class as shown below.

```
using System;
namespace LINQDemo
{
    public class Student : IEquatable<Student>
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public int TotalMarks { get; set; }
        public bool Equals(Student obj)
        {
            return this.ID == obj.ID && this.Name == obj.Name && this.TotalMar
obj.TotalMarks;
        }
        public override int GetHashCode()
        {
            return this.ID.GetHashCode() ^ this.Name.GetHashCode() ^
this.TotalMarks.GetHashCode();
        }
    }
}
```

Here, first, we implement the Equals method of the IEquatable interface, and then we override the GetHashCode method of the Object class. With the above changes in place, now modify the Main Method of the Program class as shown below.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<Student> students = new List<Student>()
            {
                new Student(){ID = 101, Name = "Priyanka", TotalMarks = 275},
                new Student(){ID = 102, Name = "Preety", TotalMarks = 280}
            };

            //Using Method Syntax
            var IsExistsMS = students.Contains(new Student() { ID = 101, Name = "Priyanka", TotalMarks = 275 });
            var student1 = new Student() { ID = 101, Name = "Priyanka", TotalMarks = 275 };

            //Using Query Syntax
            var IsExistsQS = (from num in students
```

```

        select num).Contains(student1);
    Console.WriteLine(IsExistsMS);
    Console.ReadKey();
}
}
}

```

Output: **True**

When should you use the LINQ Contains Method in C#?

The LINQ Contains Method in C# to determine whether a specific element exists within a collection or sequence. It's a useful method for performing simple existence checks and can be beneficial in various scenarios:

Checking for the existence of an item in a collection:

In the below example, we have a list of integers, and we use Contains to check if a specific number (3) exists in the list.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<int> numbers = new List<int> { 1, 2, 3, 4, 5 };
            int targetNumber = 3;
            bool containsTarget = numbers.Contains(targetNumber);
            Console.WriteLine($"Does the list contain {targetNumber}? {containsTarget}");
            Console.ReadKey();
        }
    }
}

```

Output: Does the list contain 3? True

Filtering elements in a sequence:

In the example below, we filter elements from a list of fruits based on whether they contain a specific substring ("an").

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo

```

```

{
    class Program
    {
        static void Main(string[] args)
        {
            List<string> fruits = new List<string> { "Apple", "Banana", "Cherry", "Elderberry" };
            string searchKeyword = "an";
            var matchingFruits = fruits.Where(fruit =>
fruit.Contains(searchKeyword)).ToList();
            Console.WriteLine($"Fruits containing '{searchKeyword}':");
            foreach (var fruit in matchingFruits)
            {
                Console.WriteLine(fruit);
            }
            Console.ReadKey();
        }
    }
}

```

Output:
Fruits containing 'an':
Banana

Searching for specific values in collections:

In the below example, we check if any elements from sourceList exist in targetList using Contains.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<int> sourceList = new List<int> { 1, 2, 3, 4, 5 };
            List<int> targetList = new List<int> { 3, 6, 9 };
            bool anyMatch = sourceList.Any(item => targetList.Contains(item));
            Console.WriteLine($"Any elements in sourceList exist in targetList? {anyMatch}");
            Console.ReadKey();
        }
    }
}

```

Output: Any elements in sourceList exist in targetList? True

Searching for elements in a database using LINQ to SQL or Entity Framework

Please note that this example assumes you have a database and Entity Framework set up. In the below example, we use Contains within an Entity Framework query to retrieve users whose names match any of the names in the targetNames list from the database.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using (var dbContext = new MyDbContext()) // Replace with your actual
DbContext
            {
                List<string> targetNames = new List<string> { "John", "Alice",
                var matchingUsers = dbContext.Users
                    .Where(user => targetNames.Contains(user.Name))
                    .ToList();
                Console.WriteLine("Users matching the target names:");
                foreach (var user in matchingUsers)
                {
                    Console.WriteLine($"{user.Id}: {user.Name}");
                }
            }
            Console.ReadKey();
        }
    }
}
```

In the next article, I will discuss [LINQ Deferred Execution vs Immediate Execution in C#](#) with Examples. I explain the LINQ Contains Method in C# with Examples in this article. I hope you understand the need and use of the LINQ Contains Method in C# with Examples.